



UCD College of Business

UCD College of Business, Global Campus

Assessment Submission Form

Student Name(s):	Zhu Tong	Chong Shi Xian	Han Bin
Student Number(s):	13895966	24285579	24285587
Assessment Title:	BBS41 Sg FT IP Group Project		
Module Code:	MIS3011S		
Module Title:	Intro to programming		

Declaration of Authorship: *I/We declare that all materials included in this assessment is the end result of my/our own work and that due acknowledgement has been given in the bibliography and references to ALL sources be they printed, electronic or personal.*

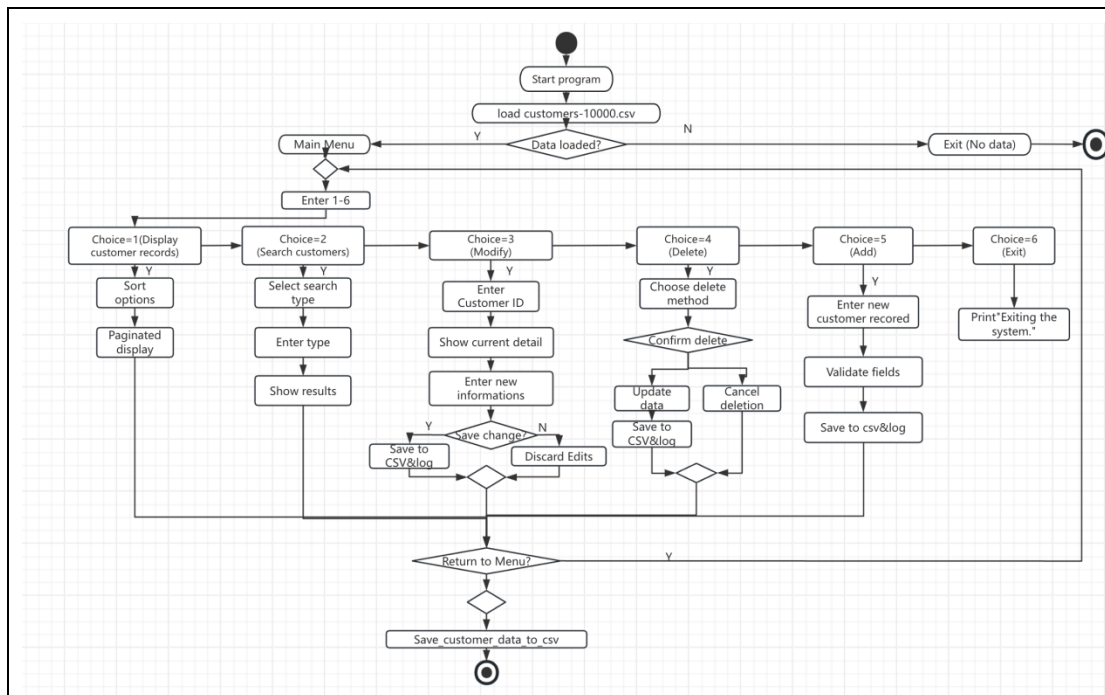
Signed:

1.Zhu Tong	2.Chong Shi Xian	3.Han Bin
------------	------------------	-----------

Date: 17 June 2025

Notes:

1. A signed copy of this form must accompany all submissions for assessment. Where this is a group assessment, all group members must sign the form.
2. Students should keep a copy of all work submitted.
3. You should submit this completed form as a separate document to your assessment (i.e. it should not be integrated within your project).
4. Procedures for Submission and Late Submission: There are penalties for the late submission of assessments. For further information, please see the University's Policy on Late Submission of Coursework.
<https://www.ucd.ie/philosophy/t4media/Application%20for%20Late%20Submission%20of%20Coursework.pdf>
5. Plagiarism: the unacknowledged inclusion of another person's writings or ideas or works, in any formally presented work (including essays, examinations, projects, laboratory reports or presentations). The penalties associated with plagiarism are designed to impose sanctions that reflect the seriousness of University's commitment to academic integrity. Ensure that you have read the below:
(a) University's Briefing for Students on Academic Integrity and Plagiarism:
https://www.ucd.ie/t4cms/UCD_Policy_on_Plagiarism_and_FAQ.pdf
(b) UCD Plagiarism Statement, Plagiarism Policy and Procedures:
https://hub.ucd.ie/usis!/W_HU_MENU.P_PUBLISH?p_tag=GD-DOCLAND&ID=222



```
# =====
```

```
# Customer Management System (CRM)
```

```
# Features: Can Load/Save, Search, Add/Edit/Delete, Logging, Pagination.
```

```
# =====
```

```
#customer_management_system.py
```

```
import csv
```

```
from datetime import datetime
```

```
DATA_FILE = "customers_10000.csv"
```

```
LOG_FILE = "customer_changes.log"
```

```
# Dfault page.
```

```
RECORDS_PER_PAGE = 10
```

```
def main():
```

```
    # Main function that run the Customer Management System.
```

```
    print("-----Welcome to the Customer Management System-----")
```

```
    customers = load_file()
```

```
    # Load customer records from CSV.
```

```
    if not customers:
```

```
        # Exit if no data is loaded.
```

```
        print("No customer data loaded. Exiting...")
```

```
        return
```

```
    while True:
```

```
        # Main menu loop.
```

```

show_main_menu()
choice = get_user_choice(1, 6)
#1. Option 1: Display all records
if choice == 1: # Display all customer records
    show_all_records(customers) #Display records
#2. Option 2: Search by last name, city, Country, Sub date range, Website or
companyname.
elif choice == 2:
    search_customers(customers)
# Option 3: Modify customer data.
elif choice == 3:
    modify(customers)
# Option 4: Delete a customer.
elif choice == 4:
    delete(customers)
# Option 5: Add new customer.
elif choice == 5:
    add(customers)
# Option 6: Exit and save.
elif choice == 6:
    print("Exiting the system...")
    save_customer_data_to_csv(customers)
    break

def load_file(filename=DATA_FILE):
    # Load customer data from CSV
    customers = []
    try:
        with open(filename, mode='r', newline="", encoding='utf-8') as file:
            reader = csv.reader(file)
            # Skip header row
            next(reader, None)
            for row in reader:
                # Skip rows with missing fields (need 12 fields)
                if len(row) < 12:
                    print("Skipping row due to insufficient fields:", row)
                    continue
                # Append valid record to list
                customers.append(row)
            return customers
    except Exception as e:
        print(f"Error information: {str(e)}")
        return None

```

```

def save_customer_data_to_csv(customers, filename=DATA_FILE):
    """Save the updated customer data back to the CSV file"""
    try:
        with open(filename, mode='w', newline='', encoding='utf-8') as file:
            # Write column headers to the first row.
            writer = csv.writer(file)
            # Write headers (assuming standard format)
            writer.writerow([
                "Customer Id", "Passport Number", "First Name", "Last Name",
                "Company", "City",
                "Country", "Phone 1", "Phone 2", "Email", "Subscription Date", "Website"
            ])
            writer.writerows(customers)
            print("-----Data saved successfully-----!")
    except Exception as e:
        print(f"Error saving: {str(e)}")

def show_main_menu():
    # Show main functions for main.
    print("\n===== Main Menu =====")
    print("-"*56)
    print("1. --- Display all records by sorted.---")
    print("2. --- Search customer information.---")
    print("3. --- Modify a record by Customer Id.---")
    print("4. --- Delete a record by Customer Id or last name.---")
    print("5. --- Add a new customer record.---")
    print("6. Exit the system.")
    print("=====")

def get_user_choice(min_option, max_option):
    # Get user's choice to input a valid number.
    while True:
        try:
            choice = int(input(f"Enter your choice ({min_option}-{max_option}): "))
            #convert to integer
            # Check input.
            if min_option <= choice <= max_option:
                return choice
            else:
                print(f"Please enter a number between {min_option} and {max_option}.")
        except ValueError:

```

```

        # non-integer input
        print("Invalid input. Please enter a number.")

def get_sort_key_by_last_name(customer):
    return customer[3]

def get_sort_key_by_id(customer):
    return customer[0]

def get_sort_key_by_date(customer):
    return customer[10]

def show_all_records(customers):
    # Display all customer records with sorting options.
    print("\nSort options:")
    print("1. By Last Name")
    print("2. By Customer Id")
    print("3. By Subscription Date")
    sort_choice = get_user_choice(1, 3)

    # Sort the customers based on choice.
    if sort_choice == 1:
        sorted_customers = sorted(customers, key=get_sort_key_by_last_name)
        # [0] customer ID ,[1] Passport,[2] First name,[3] Last name....
    elif sort_choice == 2:
        sorted_customers = sorted(customers, key=get_sort_key_by_id)
        # Customer id [0].
    else:
        # Subscription Date is [9]
        sorted_customers = sorted(customers, key=get_sort_key_by_date)

    # Display the sorted records with pagination.
    paginate_customers_records(sorted_customers)

def paginate_customers_records(records):
    # Display customer records with pagination.
    # Ensure fixed column width.
    columns = [
        ("No.", 4, None),          # row number
        ("ID", 12, 0),             # customer id
        ("First", 14, 2),          # first name
        ("Last", 12, 3),           # last name
        ("Company", 30, 4),        # company name
    ]

```

```

    ("City", 15, 5),          # city
    ("Country", 12, 6),      # country
    ("Phone1", 20, 7),       # Phone 1
    ("Phone2", 20, 8),       # Phone2
    ("Email", 30, 9),        # email
    ("SubDate", 10, 10),     # Sub Date
    ("Website", 40, 11)      # Website
]

total_pages = (len(records) + RECORDS_PER_PAGE - 1) // RECORDS_PER_PAGE
current_page = 1

while True:
    # Calculate page range.
    start_index = (current_page - 1) * RECORDS_PER_PAGE
    end_index = start_index + RECORDS_PER_PAGE
    page_records = records[start_index:end_index]
    # Display page header.
    print(f"\nPage {current_page} of {total_pages}")
    print("=" * 300)
    print(f" | {'No.':<4} | {'Customer ID':<12} | {'Passport No.':<14} | {'First Name':<12} | "
          f"{'Last Name':<12} | "
          f"{'Company':<30} | {'City':<15} | {'Country':<12} | {'Phone 1':<20} | {'Phone "
          f"2':<20} | "
          f"{'Email':<30} | {'Sub Date':<10} | {'Website':<40} | ")
    print("-" * 300)

    for i in range(len(page_records)):
        # Display records.
        record = page_records[i]
        row_no = start_index + i + 1 # display index number
        email = record[9][:24] + '...' if len(record[9]) > 25 else record[9]
        website = record[11][:29] + '...' if len(record[11]) > 30 else record[11]

        print(f" | {row_no:<4} | {record[0]:<12} | {record[1]:<14} | {record[2]:<12} | "
              f"{'record[3]:<12} | {record[4]:<30} | {record[5]:<15} | {record[6]:<12} | "
              f"{'record[7]:<20} | {record[8]:<20} | {email:<30} | {record[10]:<10} | "
              f"{'website:<40} | ")

    print("=" * 300)

    # User input navigation choice.
    navigation_choice = input("Enter 'n' (next), 'p' (previous), 'q' (quit), or number to
view full details: ")

```

```

if navigation_choice == 'n' and current_page < total_pages:
    current_page += 1
elif navigation_choice == 'p' and current_page > 1:
    current_page -= 1
elif navigation_choice == 'q':
    break
elif navigation_choice.isdigit():
    index = int(navigation_choice)
    if 1 <= index <= len(records):
        record = records[index - 1]
        print("\n***** Full Customer Details *****")
        print(f"Customer ID      : {record[0]}")
        print(f"Passport Number : {record[1]}")
        print(f"First Name       : {record[2]}")
        print(f>Last Name      : {record[3]}")
        print(f"Company          : {record[4]}")
        print(f"City             : {record[5]}")
        print(f"Country          : {record[6]}")
        print(f"Phone 1          : {record[7]}")
        print(f"Phone 2          : {record[8]}")
        print(f>Email            : {record[9]}")
        print(f"Subscription     : {record[10]}")
        print(f"Website          : {record[11]}")
        print("-----\n")
    else:
        print("Invalid number!! Please enter a valid record [number].")
else:
    print("Invalid input!! Please enter n, p, q or a [number].")

```

```

def modify(customers_list):
    # Find customer
    customer_id = input("Enter Customer Id to modify: ").strip()

    # Find the customer with matching ID
    found = None
    for customer in customers_list:
        if customer[0] == customer_id:
            found = customer
            break

    if not found:
        print("Customer ID not found.")

```

```
        return

    # Show the current details
    print("\nCurrent customer details:")
    display_customer_details(found)

    # Save the old info for comparison
    old_data = found.copy()
    print("\n--- Modify Customer Fields ---")
    print("***** -Note! Press ENTER to keep the old value. -*****\n")

    new_passport = input(f"Passport Number [{found[1]}]: ").strip()
    if new_passport != "":
        found[1] = new_passport

    new_first_name = input(f"First Name [{found[2]}]: ").strip()
    if new_first_name != "":
        found[2] = new_first_name

    new_last_name = input(f"Last Name [{found[3]}]: ").strip()
    if new_last_name != "":
        found[3] = new_last_name

    new_company = input(f"Company [{found[4]}]: ").strip()
    if new_company != "":
        found[4] = new_company

    new_city = input(f"City [{found[5]}]: ").strip()
    if new_city != "":
        found[5] = new_city

    new_country = input(f"Country [{found[6]}]: ").strip()
    if new_country != "":
        found[6] = new_country

    new_phone1 = input(f"Phone 1 [{found[7]}]: ").strip()
    if new_phone1 != "":
        found[7] = new_phone1

    new_phone2 = input(f"Phone 2 [{found[8]}]: ").strip()
    if new_phone2 != "":
        found[8] = new_phone2

    new_email = input(f"Email [{found[9]}]: ").strip()
```

```

if new_email != "":
    found[9] = new_email

new_website = input(f"Website [{found[11]}]: ").strip()
if new_website != "":
    found[11] = new_website

# Show the changes
print("\n--- Summary of Changes ---")
changed = False
for i in [1, 2, 3, 4, 5, 6, 7, 8, 9, 11]:
    if old_data[i] != found[i]:
        print("Field:", field_name(i))
        print("Old :", old_data[i])
        print("New :", found[i])
        print("-" * 30)
        changed = True

if not changed:
    print("No changes made.")
    return

# Ask user to confirm the change
confirm = input("Save these changes? (Y/N): ").strip().lower()
if confirm == "y":
    log_change("MODIFY", found)
    save_customer_data_to_csv(customers_list)
    print("Customer record updated successfully.")
else:
    # If user says no, restore old data
    for i in [1, 2, 3, 4, 5, 6, 7, 8, 9, 11]:
        found[i] = old_data[i]
    print("Changes discarded.")

# Save updates to CSV

def delete(customers):
    # Let user delete a record by Customer Id or Last Name
    print("\nDelete by:")
    print("1. Customer Id")
    print("2. Last Name")

    delete_choice = get_user_choice(1, 2)

    if delete_choice == 1:

```

```

# Option 1: Delete by Customer Id
customer_id = input("Enter Customer Id to delete: ").strip()
for i in range(len(customers)):
    if customers[i][0] == customer_id:
        # Found the customer, show details first
        print("\n=====The following is the customer information=====.")
        display_customer_details(customers[i])

        # Ask for confirmation before deleting
        confirm = input("Are you sure you want to delete this customer? (Y/N): ").strip().lower()

        if confirm == "y":
            log_change("DELETE", customers[i])
            del customers[i]
            save_customer_data_to_csv(customers)
            print("Customer deleted successfully.")
        else:
            print("Deletion cancelled.")
        return
    print("Customer ID not found.")

else:
    # Option 2: Delete by Last Name
    last_name = input("Enter Last Name to delete: ").strip()
    matches = []

    # Collect all matching records (ignore case)
    for i in range(len(customers)):
        if customers[i][3].lower() == last_name.lower():
            matches.append(i)

    if not matches:
        print("No customers found with that last name.")
        return

    if len(matches) > 1:
        # More than one match found
        print(f"Found {len(matches)} customers with last name '{last_name}':")
        for i in matches:
            print("-" * 40)
            display_customer_details(customers[i])

        customer_id = input("Enter the Customer Id of the one to delete: ").strip()
        for i in matches:

```

```

        if customers[i][0] == customer_id:
            print("\nSelected customer:")
            display_customer_details(customers[i])
            confirm = input("Are you sure you want to delete this customer?
(Y/N): ").strip().lower()
            if confirm == "y":
                log_change("DELETE", customers[i])
                del customers[i]
                save_customer_data_to_csv(customers)
                print("Customer deleted successfully.")
            else:
                print("Request for deletion cancelled.")
            return
        print("No matching Customer Id found.")

    else:
        # Only one match found, delete directly after confirmation
        print("\nFound one match:")
        display_customer_details(customers[matches[0]])
        confirm = input("Are you sure you want to delete this customer? (Y/N):
").strip().lower()
        if confirm == "y":
            log_change("DELETE", customers[matches[0]])
            del customers[matches[0]]
            save_customer_data_to_csv(customers)
            print("Customer deleted successfully.")
        else:
            print("Request for deletion cancelled.")

def add(record):
    # Add a new customer to the list.

    print("\n===== Add New Customer Record =====")
    # Get all current IDs to avoid duplicates
    existing_ids = []
    for customer in record:
        existing_ids.append(customer[0])
    # Ask user to input a new ID.
    while True:
        customer_id = input("Customer Id (must be unique): ").strip()
        if not customer_id:
            print("Customer Id cannot be empty.")
            continue
        if customer_id in existing_ids:

```

```

        print("This Customer Id already exists. Please choose another,Thank You!")
        continue
    break

# Get the rest of the informations.
passport_number = get_non_empty_input("Passport Number: ")
first_name = get_non_empty_input("First Name: ")
last_name = get_non_empty_input("Last Name: ")
company = get_non_empty_input("Company: ")
city = get_non_empty_input("City: ")
country = get_non_empty_input("Country: ")
phone1 = get_non_empty_input("Phone 1: ")
phone2 = input("Phone 2 (optional): ").strip()
email = valid_email()
subscription_date = input("Enter Subscription Date (format: YYYY/M/D): ")
website = get_non_empty_input("Website: ")

# Fill in everything into a new list.
new_customer = [
    customer_id, passport_number, first_name, last_name, company, city, country,
    phone1, phone2, email, subscription_date, website
]

# Generate a new records and sav.
record.append(new_customer)          # Add to the list
log_change("ADD", new_customer)      # Log the new customer
print("-----New customer added successfully!-----")
save_customer_data_to_csv(record)    # Save changes to CSV

def get_non_empty_input(prompt):
    """Ensure the response is not empty"""
    while True:
        value = input(prompt).strip()
        if value:
            return value
        print("This field cannot be empty.")

def valid_email():
    """Require input valid email format"""
    while True:
        email = input("Email: ")
        if "@" in email and "." in email:
            return email
        print("Please enter a valid email address.")

```

```

def display_customer_details(customer):
    # Display detailed information.
    print("\nCustomer Details:")
    print(f"Customer Id: {customer[0]}")
    print(f"Passport Number: {customer[1]}")
    print(f"Name: {customer[2]} {customer[3]}")
    print(f"Company: {customer[4]}")
    print(f"Location: {customer[5]}, {customer[6]}")
    print(f"Phone 1: {customer[7]}")
    print(f"Phone 2: {customer[8]} if customer[8] else 'N/A'")
    print(f"Email: {customer[9]}")
    print(f"Subscription Date: {customer[10]}")
    print(f"Website: {customer[11]}")

def field_name(index):
    # Return field name by index for display (for printing changes)
    names = [
        "Customer Id", "Passport Number", "First Name", "Last Name", "Company",
        "City", "Country", "Phone 1", "Phone 2", "Email", "Subscription Date", "Website"
    ]
    return names[index]

def log_change(action, customer):
    # Save a log entry when we add, change, or delete a customer.
    try:
        with open(LOG_FILE, 'a', encoding='utf-8') as log_file:
            timestamp = datetime.now().strftime("%Y/%m/%d %H:%M:%S") # store real
time
            log_file.write(f"{timestamp} - {action} - ID: {customer[0]}, Name: {customer[2]}
{customer[3]}, Email: {customer[9]}\n")
    except Exception as e:
        print(f"-----Warning: Could not log change----- {str(e)}")

def search_customers(customers):
    # Check if there are any customer records.
    if not customers:
        print("No customer records to search.")
        return

    # Show search options
    print("\n-----Search options-----")
    print("1. By Last Name")
    print("2. By City")
    print("3. By Country")

```

```

print("4. By Subscription Date Range")
print("5. By Website")
print("6. By Company Name")
choice = input("Select search type (1-6): ")

results = []

# 1. Search by last name
if choice == '1':
    search_term = input("Please enter last name: ")
    for c in customers:
        if search_term in c[3]:
            results.append(c)

# 2. Search by city
elif choice == '2':
    search_term = input("Please enter city: ")
    for c in customers:
        if search_term in c[5]:
            results.append(c)

# 3. Search by country
elif choice == '3':
    search_term = input("Please enter country: ")
    for c in customers:
        if search_term in c[6]:
            results.append(c)

# 4. Search by subscription date (between two dates)
elif choice == '4':
    print("Search by subscription date range (format: YYYY/M/D)")
    start_date = input("Start date: ")
    end_date = input("End date: ")
    for c in customers:
        sub_date = c[10]
        if start_date <= sub_date <= end_date:
            results.append(c)

# 5. Search by part of website
elif choice == '5':
    search_term = input("Please enter part of website: ")
    for c in customers:
        if search_term in c[11]:
            results.append(c)

# 6. Search by part of company name
elif choice == '6':
    search_term = input("Please enter part of company name: ")
    for c in customers:
        if search_term in c[4]:

```

```
        results.append(c)

    else:
        print("Invalid choice.") # Wrong input
        return

    # show result
    if len(results) == 0:
        print("No matching records found.")
    else:
        print("Found", len(results), "matching records:")
        paginate_customers_records(results)

main()
```